

Análisis técnico de Apache Cassandra como tecnología de base de datos distribuida

José Alejandro Lozano Morales
Informe Técnico Individual
APLICACIONES DISTRIBUIDAS

Resumen

Apache Cassandra es una base de datos NoSQL distribuida orientada al manejo de grandes volúmenes de información, alta disponibilidad y escalabilidad horizontal. Su arquitectura permite distribuir datos entre varios nodos, replicar información y mantener el servicio activo incluso ante fallos parciales del sistema. Este informe analiza sus principales características técnicas, su relación con el teorema CAP, su modelo de consistencia y algunos casos de uso documentados en investigaciones recientes. Además, se evalúa su pertinencia para proyectos que requieren almacenamiento distribuido, procesamiento de grandes volúmenes de datos y despliegues en entornos modernos como contenedores o infraestructura cloud.

1. Introducción

En los últimos años, el crecimiento de los datos ha impulsado el uso de tecnologías capaces de almacenar, distribuir y procesar información en múltiples servidores. Las bases de datos relacionales continúan siendo importantes, pero en escenarios de Big Data, analítica en tiempo real, Internet de las Cosas y aplicaciones con alto tráfico, las bases de datos NoSQL se han vuelto una alternativa relevante. Dentro de este grupo, Apache Cassandra destaca por su arquitectura distribuida, su capacidad de escalar horizontalmente y su diseño orientado a la disponibilidad.

Apache Cassandra fue diseñada para trabajar en clústeres de múltiples nodos, evitando depender de un único servidor central. Esto significa que la información puede repartirse entre varias máquinas y replicarse para mejorar la tolerancia a fallos. A diferencia de una base de datos tradicional con arquitectura cliente-servidor centralizada, Cassandra utiliza un modelo descentralizado donde todos los nodos pueden cumplir funciones similares.

Esta característica permite que el sistema siga funcionando aunque uno o varios nodos presenten fallos.

La literatura reciente muestra que Cassandra es utilizada tanto como objeto de investigación como en casos reales relacionados con rendimiento, sistemas distribuidos, almacenamiento de grandes datos y despliegue en infraestructuras modernas. Por ejemplo, Eppinger y Störl estudian la optimización del rendimiento de Cassandra mediante aprendizaje automático, considerando variables como configuración, replicación y particionamiento [1]. Fleming et al. presentan Fallout, un servicio usado por DataStax para realizar pruebas automatizadas sobre sistemas distribuidos como Apache Cassandra [2].

2. Características de distribución de datos

Una de las principales características de Apache Cassandra es su forma de distribuir datos. Cassandra organiza la información mediante una arquitectura de tipo *peer-to-peer*, donde no existe un nodo maestro único. Cada nodo del clúster puede recibir solicitudes de lectura o escritura, y el sistema se encarga de ubicar los datos correspondientes dentro del conjunto de nodos disponibles.

La distribución se basa en el particionamiento de datos. Cada registro se asocia a una clave de partición, y mediante un mecanismo de hashing se determina en qué nodo o nodos debe almacenarse. Esta estrategia permite repartir la carga entre diferentes servidores, evitando que una sola máquina concentre todo el trabajo. A medida que el volumen de datos crece, es posible añadir nuevos nodos al clúster para incrementar la capacidad total del sistema.

Además del particionamiento, Cassandra utiliza replicación. Esto significa que una misma información puede guardarse en más de un nodo. La replicación mejora la disponibilidad, porque si un nodo deja de funcionar, otro nodo con una copia de los datos puede responder a la solicitud. Esta característica es especialmente importante en aplicaciones donde el sistema debe mantenerse activo de forma continua.

El trabajo de Canon et al. analiza a Cassandra como un almacén persistente de clave-valor distribuido, donde la replicación permite seleccionar qué réplica debe atender una solicitud. Este tipo de decisiones influye directamente en el rendimiento, la latencia y el throughput del sistema [3]. Por tanto, la distribución de datos en Cassandra no solo sirve para almacenar grandes volúmenes de información, sino también para mejorar el comportamiento del sistema bajo alta demanda.

3. Modelo de consistencia y relación con CAP

El teorema CAP establece que en un sistema distribuido no se pueden garantizar simultáneamente y de forma absoluta tres propiedades: consistencia, disponibilidad y

tolerancia a particiones. La consistencia implica que todos los nodos observan el mismo dato actualizado al mismo tiempo. La disponibilidad significa que el sistema responde a las solicitudes aunque existan fallos. La tolerancia a particiones indica que el sistema puede continuar funcionando aunque existan problemas de comunicación entre nodos.

Apache Cassandra suele clasificarse como una base de datos orientada principalmente a disponibilidad y tolerancia a particiones. Esto se debe a que su diseño busca que el sistema continúe operando aunque algunos nodos fallen o la red presente divisiones. Sin embargo, Cassandra no ignora por completo la consistencia. En lugar de imponer un único nivel fijo, ofrece consistencia configurable. Esto permite decidir cuántas réplicas deben confirmar una operación antes de considerarla exitosa.

Por ejemplo, en una operación de escritura, se puede configurar un nivel de consistencia bajo para obtener mayor velocidad y disponibilidad, o un nivel más alto para exigir confirmación de más nodos. Esta flexibilidad permite adaptar Cassandra según las necesidades del sistema. En aplicaciones donde se prioriza la rapidez y la disponibilidad, se puede aceptar una consistencia eventual. En cambio, en escenarios donde los datos deben ser más estrictos, se puede aumentar el nivel de consistencia.

La consistencia eventual significa que, después de una actualización, no todos los nodos tienen necesariamente el valor más reciente de forma inmediata, pero el sistema terminará propagando los cambios hasta que las réplicas converjan. Este modelo es adecuado para aplicaciones distribuidas donde se acepta un pequeño retraso en la sincronización a cambio de mayor disponibilidad y escalabilidad.

4. Casos de uso reales documentados

Cassandra aparece en diferentes investigaciones recientes como tecnología de almacenamiento distribuido. Un caso relevante es el trabajo de Benítez-Hidalgo et al., donde se presenta NORA, un razonador OWL escalable basado en Apache Spark y bases de datos NoSQL. En este caso, Cassandra se utiliza para materializar datos OWL, jerarquías, propiedades e instancias, permitiendo trabajar con bases de conocimiento de gran tamaño [4]. Este ejemplo muestra la utilidad de Cassandra en escenarios donde se necesita persistencia distribuida para datos complejos.

Otro caso documentado se encuentra en el trabajo de Sepulveda et al., quienes presentan una arquitectura de almacenamiento y cómputo en el borde para redes de sensores IoT. El estudio utiliza Cassandra dentro de un marco orientado a adquirir, transmitir, almacenar y procesar grandes volúmenes de datos sísmicos generados por sensores [5]. Este caso resulta importante porque demuestra que Cassandra puede aplicarse en contextos donde los datos se generan constantemente y deben almacenarse cerca del lugar donde se producen.

También existen estudios enfocados en su despliegue en infraestructura virtualizada y

contenedores. Shirinbab et al. comparan el rendimiento de Cassandra ejecutándose sobre infraestructura física, máquinas virtuales VMware y contenedores Docker. Los resultados indican que Docker presenta menor sobrecarga que VMware y que el rendimiento en contenedores puede acercarse al de la infraestructura no virtualizada [6]. Esto es relevante porque muchos proyectos actuales se implementan mediante contenedores para facilitar despliegue, portabilidad y administración.

Finalmente, Babaee et al. comparan Apache Cassandra con Aerospike desde la perspectiva de eficiencia energética y emisiones de CO₂ [7]. Aunque este enfoque no se centra únicamente en distribución o consistencia, aporta una visión moderna sobre requisitos no funcionales, donde el consumo energético y el impacto ambiental también pueden influir en la selección de una tecnología de base de datos.

5. Pertinencia para el proyecto TiendaTech

Apache Cassandra puede ser pertinente para TiendaTech como una base de datos distribuida complementaria, especialmente en los módulos donde se generen grandes volúmenes de información y se requiera alta disponibilidad. El proyecto plantea una plataforma de comercio electrónico basada en microservicios para autenticación, catálogo, carrito, inventario, pedidos, pagos, notificaciones, motor de compatibilidad de hardware y asistente inteligente. Por ello, no todos los datos tienen las mismas necesidades de almacenamiento.

En este contexto, Cassandra sería útil para almacenar eventos del sistema, logs de actividad, historial de consultas de compatibilidad, métricas del asistente inteligente, movimientos de inventario y registros de navegación. Estos datos pueden crecer rápidamente y se benefician de una arquitectura escalable y tolerante a fallos. Además, como TiendaTech contempla el uso de Docker y Docker Compose, Cassandra podría integrarse como un servicio adicional dentro del ecosistema distribuido.

Sin embargo, Cassandra no debería reemplazar completamente a una base relacional. Módulos como usuarios, pagos, facturación y pedidos confirmados requieren mayor consistencia transaccional, relaciones claras y restricciones de integridad. Para estos casos, una base como PostgreSQL seguiría siendo más adecuada.

Por tanto, la mejor forma de incorporar Cassandra en TiendaTech sería mediante un enfoque híbrido: mantener la base relacional para los procesos críticos y utilizar Cassandra para datos masivos, históricos, eventos y consultas de alta disponibilidad. Así, el proyecto aprovecharía la escalabilidad distribuida de Cassandra sin perder la consistencia necesaria en las operaciones comerciales principales.

6. Conclusiones

Apache Cassandra es una tecnología adecuada para proyectos que priorizan disponibilidad, escalabilidad y tolerancia a fallos. Su arquitectura descentralizada permite distribuir datos entre múltiples nodos, replicar información y mantener el servicio activo incluso cuando existen fallos parciales. Desde la perspectiva del teorema CAP, Cassandra se orienta principalmente hacia disponibilidad y tolerancia a particiones, aunque permite ajustar el nivel de consistencia según las necesidades del sistema.

Los casos reales documentados muestran que Cassandra se utiliza en escenarios relacionados con Big Data, IoT, procesamiento distribuido, pruebas de sistemas y análisis de rendimiento. Por ello, representa una opción sólida cuando el proyecto necesita una base de datos distribuida capaz de manejar grandes volúmenes de información de manera flexible. No obstante, debe elegirse con cuidado, ya que no reemplaza completamente a una base de datos relacional cuando el sistema requiere muchas relaciones, transacciones estrictas o consultas complejas.

Referencias

- [1] F. Eppinger y U. Störl, «Tuning Cassandra through Machine Learning,» en *Datenbanksysteme für Business, Technologie und Web (BTW 2023), Lecture Notes in Informatics*, Gesellschaft für Informatik, 2023. DOI: [10.18420/BTW2023-04](https://doi.org/10.18420/BTW2023-04).
- [2] M. Fleming, G. B. King, S. McCarthy, J. Luciani y P. Pattabhiraman, «Fallout: Distributed systems testing as a service,» *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, vol. 1, pág. 100 010, 2021. DOI: [10.1016/j.tbench.2021.100010](https://doi.org/10.1016/j.tbench.2021.100010).
- [3] L.-C. Canon, A. Dugois, L. Marchal y E. Rivière, «Hector: A Framework to Design and Evaluate Scheduling Strategies in Persistent Key-Value Stores,» en *Proceedings of the 52nd International Conference on Parallel Processing*, ACM, 2023, págs. 535-545. DOI: [10.1145/3605573.3605614](https://doi.org/10.1145/3605573.3605614).
- [4] A. Benítez-Hidalgo, I. Navas-Delgado y M. d. M. Roldán-García, «NORA: Scalable OWL reasoner based on NoSQL databases and Apache Spark,» *Software: Practice and Experience*, vol. 53, págs. 2377-2392, 2023. DOI: [10.1002/spe.3258](https://doi.org/10.1002/spe.3258).
- [5] F. Sepulveda, J. S. Thangraj y J. Pulliam, «The Edge of Exploration: An Edge Storage and Computing Framework for Ambient Noise Seismic Interferometry Using Internet of Things Based Sensor Networks,» *Sensors*, vol. 22, n.º 10, pág. 3615, 2022. DOI: [10.3390/s22103615](https://doi.org/10.3390/s22103615).

- [6] S. Shirinbab, L. Lundberg y E. Casalicchio, «Performance evaluation of containers and virtual machines when running Cassandra workload concurrently,» *Concurrency and Computation: Practice and Experience*, vol. 32, n.º 16, e5693, 2020. DOI: [10.1002/cpe.5693](https://doi.org/10.1002/cpe.5693).
- [7] B. Babaei, D. Daylamani-Zad y K. Tune, «CO2 Emission Efficiency as a Measurable Non-Functional Requirement: An Emission Estimation Framework,» *IEEE Access*, vol. 10, págs. 97 576-97 587, 2022. DOI: [10.1109/ACCESS.2022.3205016](https://doi.org/10.1109/ACCESS.2022.3205016).